

AI Reference Report

Project: Mario-style Platformer (HW2)
Engine: Cocos Creator 2.4.8
Language: TypeScript

1. AI Tools Used

Tool	Model	Purpose
Claude Code (Anthropic)	Claude Sonnet 4.6	Architecture guidance, refactoring, debugging, deployment

Claude Code was used interactively through the CLI during development. It was not used to generate the project from scratch, but rather as an assistant for code review, refactoring guidance, identifying bugs, and explaining Cocos Creator 2.4.x-specific behavior that was not well-documented.

2. Scope of AI-Assisted Development

2.1 Movement Bitflag System

Files: `assets/Function/MovementFlags.ts` (lines 1-43),
`assets/Script/PlayerController.ts` (lines 29, 129-170)

Prompt used:

"I want to track player movement direction using bitflags so I can represent combined states like UP+LEFT or UP+RIGHT in a single variable. Help me design a clean enum and helper functions for this."

Summary of AI response:

Claude suggested using a bitflag enum where IDLE = 0x0, LEFT = 0x1, RIGHT = 0x2, and UP = 0x4. This allows combined states like UP_LEFT (0x5) and UP_RIGHT (0x6) to be read directly from the enum. It also generated standalone `beginMoveLeft`, `endMoveLeft`, `beginMoveRight`, `endMoveRight`, `beginJump`, and `endJump` functions that operate on a shared `MovementRef` interface, so the same functions could be reused for any player object.

My modifications:

I added the `stopAllMovement` utility function separately and defined the `MovementRef` interface to be implemented by `PlayerController`, which was not part of the original suggestion. I also verified the bitflag values manually to confirm that no two directional combinations would produce the same integer.

How it works:

Each direction is a power of two. Pressing left sets bit 0, pressing right sets bit 1, pressing jump sets bit 2. The movement switch statement in `updateMovement()` reads the combined enum value and handles UP_LEFT and UP_RIGHT as fall-through cases — jump is attempted first, then horizontal velocity is applied. `endMoveRight` is called inside `beginMoveLeft` and vice versa to prevent simultaneous left+right inputs, which would otherwise produce an invalid state.

2.2 Player Physics and Collision Callbacks

Files: `assets/Script/PlayerController.ts` (lines 352–445)

Prompt used:

"How do I detect whether the player is landing from above versus colliding from the side using Cocos Creator 2.4.8's physics contact callbacks? I need to support pass-through platforms and track ground contact for jump gating."

Summary of AI response:

Claude explained that `getWorldManifold().normal.y` returns -1 when a surface is directly below the player (contact normal pointing downward into the player), and 1 when the player is below a surface. It outlined the three callbacks — `onBeginContact`, `onPreSolve`, and `onEndContact` — and explained that `onPreSolve` is called every physics step and is the correct place to disable contacts conditionally. It also suggested a `groundContactCount` counter rather than a boolean to correctly handle cases where the player stands on the edge of two ground tiles simultaneously.

My modifications:

The one-way platform logic required careful handling in both `onBeginContact` and `onPreSolve`. I observed that the contact normal check alone was insufficient because the player's horizontal velocity could trigger a side-contact event on a pass-through platform. I added an explicit check for `Math.abs(normal.x) !== 0` to catch those cases. I also added the `isStandingOnPassThrough` flag to prevent `groundContactCount` from incrementing twice on the same platform contact (once in `onBeginContact` and once in `onPreSolve`).

How it works:

`groundContactCount` is incremented on downward ground contact and decremented on contact

end. Jump is only permitted when this count is greater than zero. Pass-through platforms are handled by disabling the contact when the player is approaching from below or from the side, and only allowing it to resolve when the player is descending onto the platform from above.

2.3 Enemy Wall Detection Fix

Files: `assets/Script/StageController.ts` (lines ~500–700, enemy init sections)

Prompt used:

"My Goomba enemies stop moving when they hit a wall – they detect the wall contact but sometimes the normal.y check isn't triggering correctly. What's the safer way to check for wall versus floor contacts?"

Summary of AI response:

Claude identified that using `=== 1` for a floating-point normal comparison is unreliable due to physics engine precision. It recommended using a threshold comparison such as `> 0.9` instead, which correctly identifies vertical contacts (floor/ceiling) while treating near-horizontal normals as wall contacts.

My modifications:

I applied the fix specifically to the wall-detection branch inside the enemy movement callbacks. I also verified manually in-game that enemies turn around at barriers and do not get stuck on ground tiles, which required a small adjustment to the direction-multiplication guard that checks whether the enemy is already moving away from the wall.

How it works:

Each enemy's rigid body has an `onBeginContact` callback assigned at runtime in `StageController`. When a contact normal's Y component is above 0.9 (essentially vertical), it is treated as a floor contact and ignored for wall-turning purposes. When the Y component is below 0.9, it is treated as a wall and the enemy's horizontal velocity is reversed.

2.4 BigMario Death Crash Fix

Files: `assets/Script/PlayerController.ts` (lines 286–296)

Prompt used:

"My game crashes when a BigMario player enters the death zone. The crash points to an animation lookup for 'BigMarioDead'. There is no BigMarioDead animation in the project."

Summary of AI response:

Claude traced the crash path: `playDeathAnimation()` calls `playAnimation('MarioDead')`, which internally prefixes 'Big' when `isBigMario` is true, producing 'BigMarioDead'. Since that animation asset does not exist in the project, the engine throws an error. The fix is to clear `isBigMario` before the animation lookup so the death sequence always uses 'MarioDead' regardless of power-up state.

My modifications:

I added `this.isBigMario = false;` as the first line of `playDeathAnimation()`. I also confirmed this was the right placement rather than inside `playDeathAndRespawn()` because `playDeathAnimation` is called directly from `StageController` for the game-over path (without respawn), meaning the fix needed to live in the base method.

How it works:

By resetting `isBigMario` before any animation is played, the `playAnimation` helper always selects the small Mario animation variants. The collider size and offset changes from the power-up are not reversed in this path (they reset on respawn), but that is acceptable because the player is either respawning or the game is ending.

2.5 Camera Follow System

Files: `assets/Script/FollowCamera.ts` (lines 1–38)

Prompt used:

"Help me write a smooth follow camera for Cocos Creator 2.4.8 that tracks the active player, clamps to $x \geq 0$, and can be switched between players in local multiplayer."

Summary of AI response:

Claude provided a lerp-based follow implementation using `Vec2.lerp` with a factor of 0.2 per frame. It included a clamp to prevent the camera from scrolling left of the world origin and a `setFollowTarget()` method to switch between players.

My modifications:

I added a `yAxisValid` inspector property to toggle vertical tracking, since Stage 1 is a flat scrolling level where Y tracking is undesirable, while Stage 2 uses vertical sections. I also added a guard to skip the follow update while the player's death animation is playing, to prevent the camera from following the death bounce offscreen.

How it works:

Each frame, the camera's current position is lerped toward the target player's position at a rate of 0.2,

producing smooth following. The X position is clamped to zero so the background does not scroll into negative space. Y tracking is conditional on the `yAxisValid` flag set in the Inspector per scene.

2.6 Pause System

Files: `assets/Script/PausableBody.ts` (lines 1–62),
`assets/Script/StageController.ts` (lines 287–291)

Prompt used:

"What's the best way to pause physics objects in Cocos Creator 2.4.8 without using `cc.director.pause`, which pauses everything including my pause menu UI?"

Summary of AI response:

Claude explained that `cc.director.pause()` halts the entire scheduler, which breaks any UI that needs to respond during pause. The alternative is to individually deactivate each rigid body's physics simulation, save its velocity, and lock its position in the `update` loop. It provided a `PausableBody` component pattern where `togglePause()` stores and restores velocity and uses `rigidBody.active = false` to suspend physics.

My modifications:

I added animation pausing/resuming via `cc.Animation.pause()` and `cc.Animation.resume()`. I also added a one-frame delay when re-activating the rigid body (using `scheduleOnce` with no delay argument) to avoid stale contact forces from the previous frame causing objects to lurch on resume.

How it works:

`StageController` maintains a `pausableComponents` array populated at initialization with every `PausableBody` in the scene. When `togglePause()` is called on the controller, it iterates that array and calls `togglePause()` on each component. Each component saves its rigid body velocity, sets `linearVelocity` to zero, deactivates the body, and locks the node position in `update`. On resume, it restores the saved velocity and reactivates the body on the next frame.

2.7 Scene and Game Mode Management

Files: `assets/Script/StageController.ts` (lines 38–40, 142–180),
`assets/Script/StageSelectScreen.ts` (lines 31–54),
`assets/Script/StageLoadingScreen.ts` (lines 31–63)

Prompt used:

"How do I pass data between scenes in Cocos Creator 2.4.8? I need to communicate the selected stage number and game mode (single player or local multiplayer with N players) from the stage select screen to the stage scene."

Summary of AI response:

Claude recommended using static class properties on a persistent manager class since Cocos Creator 2.4.x does not have a built-in scene parameter mechanism. It suggested defining `static stageChoice` and `static playMode` on `StageController`, setting them before calling `cc.director.loadScene()`, and reading them in the destination scene's `onLoad` or `start`.

My modifications:

I typed the `playMode` property as a discriminated union (`{ mode: 'Single' } | { mode: 'LocalMultiple', payload: number } | ...`) rather than a plain object, which makes the multiplayer player count available only when the mode is `LocalMultiple`. I added the `LoadStage` intermediate scene to show a loading screen rather than jumping directly into the stage, and the loading screen validates that `stageChoice` is set before proceeding.

How it works:

`StageSelectScreen` sets the static properties then loads the `LoadStage` scene. `StageLoadingScreen` reads those properties to display appropriate UI (multiplayer player count), then loads the target stage after a short delay. `StageController.onLoad()` reads the same static properties to decide how many players to spawn and which key bindings to assign.

2.8 Local Multiplayer Key Bindings

Files: `assets/Script/StageController.ts` (lines 31–36),
`assets/Script/PlayerController.ts` (lines 52–54, 109–125)

Prompt used:

"I want to support up to 4 local players on one keyboard. How should I structure key bindings so each player has a different set of keys, and the same `PlayerController` script can be reused for all of them?"

Summary of AI response:

Claude suggested a `PlayerKeyBindings` interface with `left`, `right`, `up`, and `switchCamera` fields, and a static array of four pre-defined binding sets on the controller. It recommended a `setupAsLocalPlayer()` method that accepts bindings and a camera reference, keeping the key handling logic inside `PlayerController` rather than a global input manager.

My modifications:

I added a `switchCamera` binding to let each local player switch the shared follow camera to track their own character. I also added a `setupAsMainPlayer()` variant that does not require explicit bindings (it uses the first set by default), making the single-player path simpler to call from `StageController`.

How it works:

Each `PlayerController` stores its own `keyBindings` object. `onKeyDown` and `onKeyUp` on the controller compare against those bindings. `StageController` dispatches raw key events from the system to all registered players by iterating `playerRegistry`. Because each player's bindings are independent, there is no shared state – two players pressing different keys simultaneously do not interfere.

2.9 Score, Lives, and HUD Management

Files: `assets/Script/StageController.ts` (lines 43–46, 183–275, 370–394)

Prompt used:

"Help me implement score tracking with time bonus on stage clear, persistent high score using localStorage, and a lives counter with game over transition for a Cocos Creator 2.4.8 platformer."

Summary of AI response:

Claude provided the structure for a timer-based countdown, scoring on enemy kills and item collection, and a stage-clear score calculation that multiplies remaining timer seconds by 10 and adds them to the score. It also demonstrated `cc.sys.localStorage` for persisting the high score across sessions.

My modifications:

I broke out `handlePlayerHit` as a shared method called from multiple collision handlers (death zone, enemy contact, and direct enemy callbacks) to avoid duplicating the power-down / respawn / game-over logic. I also added `resetTimerForMode` to only reset the timer in single-player mode, since local multiplayer shares a single timer and should not reset on individual player death.

How it works:

`score` and `coinCount` are instance variables updated directly when enemies are stomped, items collected, or question boxes hit. The `update` loop writes formatted string values to the HUD labels each frame. On stage clear, `handleStageComplete` calculates the final score and saves it via `cc.sys.localStorage.setItem`. On game over, `saveHighScore` compares the current score to the stored value and updates it if higher.

2.10 Firebase Hosting Deployment

Files: `firebase.json`, `.firebaserc` (project root)

Prompt used:

"Help me deploy my Cocos Creator 2.4.8 web build to Firebase Hosting. I have a Firebase project configured. The public directory should be build/web-desktop."

Summary of AI response:

Claude provided the complete `firebase.json` configuration with MIME type headers for `.js` and `.wasm` files (to prevent browser rejection of scripts with incorrect content types), a `Cache-Control: no-cache` header to prevent stale cached assets during grading, and the correct answer to every `firebase init` prompt for a non-SPA Cocos Creator project.

My modifications:

The `.firebaserc` and `firebase.json` were created directly based on the guidance. The deployment command was adjusted to explicitly target `--project mariohw2-3d340` because the Firebase CLI was globally set to a different active project from a previous lab assignment.

How it works:

Firebase Hosting serves files from the `build/web-desktop` directory. The MD5-hashed filenames produced by Cocos Creator's build process (e.g., `main.08da6.js`) are served with `no-cache` headers, ensuring each visit fetches the latest build. Explicit content-type headers prevent Firefox and Safari from treating JavaScript files as plain text and refusing to execute them.

Deployed URL: <https://mariohw2-3d340.web.app>

2.11 Project Recovery After Hierarchy Reorganization

Context: After reorganizing the node hierarchy and renaming scripts and files in the editor, the project became unresponsive on startup (stuck at "Importing assets, please wait...").

Prompt used:

"My Cocos Creator 2.4.8 project is stuck on importing assets after I renamed scripts and reorganized the node hierarchy. How do I recover it safely?"

Summary of AI response:

Claude performed a UUID audit of all `.fire` scene files and `.meta` asset files to identify which references were genuinely broken versus which were false positives (sprite frame sub-assets and the built-in default material UUID). It confirmed that the script renames were done correctly (`.meta` files

moved alongside `.ts` files, preserving UUIDs) and that the actual root cause was a corrupted Cocos Creator library cache rather than broken serialized references.

Resolution:

The `library/` and `temp/` folders were deleted to force Cocos Creator to rebuild its import cache from scratch. On next launch, all assets were reimported cleanly and the project loaded correctly.

What I learned:

The `library/` folder is a compiled representation of all project assets — it is safe to delete because Cocos Creator regenerates it from the source files. Broken import behavior after file reorganization is almost always a cache issue rather than structural corruption.

3. Personal Modifications and Refactoring

The following changes were made manually without AI generation, based on my own understanding of the project and testing:

Script Renaming

All scripts were renamed to reflect their actual responsibilities rather than their original names:

Original Name	Renamed To	Reason
<code>Mario.ts</code>	<code>PlayerController.ts</code>	The class controls any player, not just Mario specifically
<code>StageMgr.ts</code>	<code>StageController.ts</code>	Consistent naming with Cocos component conventions
<code>PauseResume.r.ts</code>	<code>PausableBody.ts</code>	Describes what the component IS rather than what it does
<code>PlayerCamera.ts</code>	<code>FollowCamera.ts</code>	Camera can follow any target, not only the player
<code>DistantBox.ts</code>	<code>OneWayPlatform.ts</code>	The original name described nothing about the collision behavior
<code>ChooseStage.ts</code>	<code>StageSelectScreen.ts</code>	Matches the screen it controls
<code>LoadScene.ts</code>	<code>StageLoadingScreen.ts</code>	More specific to its function
<code>Enemy.ts</code>	<code>EnemyTag.ts</code>	Clarifies it is a tag/marker component, not a full controller
<code>Ground.ts</code>	<code>GroundTag.ts</code>	Same reasoning as above

Original Name	Renamed To	Reason
<code>Barrier.ts</code>	<code>BarrierTag.ts</code>	Same reasoning as above

Function File Renaming

Original Name	Renamed To
<code>BodyType.ts</code>	<code>CollisionTag.ts</code>
<code>Movement.ts</code>	<code>MovementFlags.ts</code>
<code>types.ts</code>	<code>InputBindings.ts</code>
<code>Serialize.ts</code>	<code>JsonSerializer.ts</code>

Comment Cleanup

All files were reviewed and comments were reduced to single-line notes explaining non-obvious behavior only. Multi-line descriptive comments were removed because the code itself, combined with accurate naming, communicates intent clearly enough.

Hierarchy Organization

Node groups in the Stage scenes were organized into named parent nodes: `Grounds` , `Goombas` , `Turtles` , `Flowers` , `Coins` , `CoinQuestionBoxes` , `PowerQuestionBoxes` , `HealQuestionBoxes` . This made the scene hierarchy readable and made node-group iteration in `StageController` possible without `cc.find()` .

Collision Tag Replacement

The original project used raw integers for collision identification. These were replaced with a `CollisionTag` enum, making every `switch` statement in the contact callbacks self-documenting.

4. Open-Source Reference Acknowledgement

This project was built starting from a publicly available Cocos Creator Mario platformer reference project found on GitHub. The original repository provided:

- The basic scene layout and tilemap structure
- Sprite assets and animation clips for Mario, Goomba, and Turtle

- The overall scene flow (Menu → Stage Select → Stage → Game Over)
- Initial physics collider configuration on the ground and enemy nodes

The following were significantly changed from the reference:

- All TypeScript files were rewritten or substantially refactored — none of the original script logic was kept unchanged
- The movement system was replaced with the bitflag architecture described in Section 2.1
- The collision system was restructured to use the `CollisionTag` enum and to separate ground contact counting from one-way platform logic
- Local multiplayer support was added, which was not present in the reference
- The pause system was replaced with the component-based `PausableBody` approach
- All script names, class names, and variable names were changed
- The Firebase deployment infrastructure was built entirely for this submission

The reference was used as a starting point for understanding the scene structure and asset organization in Cocos Creator 2.4.8. The final codebase does not share logic or implementation with the reference.

5. Learning and Understanding

AI assistance in this project was used the way a reference document or senior developer might be consulted — to explain engine-specific APIs, confirm whether an approach was architecturally sound, and trace the root cause of bugs that were difficult to diagnose from the error message alone.

In every case where AI provided code, I:

1. Read the generated output before using it
2. Tested it in the engine to observe the actual behavior
3. Modified it where it did not match the specific requirements of this project
4. Verified edge cases manually (jumping on two ground tiles simultaneously, BigMario death, enemy wall turning, multiplayer key isolation)

The physics contact callback system in Cocos Creator 2.4.x has limited official documentation, and AI was particularly useful for clarifying the difference between `onBeginContact`, `onPreSolve`, and `onEndContact` and when to use each. Understanding this was necessary to implement pass-through platforms correctly, which I then verified through in-editor testing.

The Firebase deployment section involved no gameplay code. AI was used to generate the correct `firebase.json` configuration and to identify why the initial deploy targeted the wrong project (a global CLI setting from a previous lab).

6. Statement of Non-Usage

The following were implemented or configured manually without AI assistance:

- **Tilemap design** – stage layouts in `stage1.tmx` and `stage2.tmx` were edited directly in the Tiled map editor
 - **Animation clips** – all `.anim` files for Mario, Goomba, and Turtle were configured manually in the Cocos Creator Animation Editor
 - **Inspector wiring** – all `@property` slots in the scene (node references, prefab references, audio clips) were connected manually by dragging in the Cocos Creator Inspector
 - **Prefab structure** – the internal node structure of each prefab (Mario, Goomba, Turtle, QuestionBox, Mushrooms, Coin) was configured manually
 - **Sprite atlas layout** – the sprite sheet organization in `effects_UI_tiles/` was not modified; original atlas files were used as-is
 - **Scene hierarchy layout** – the arrangement of UI nodes (InfoBar, Buttons, LevelClear overlay) was done manually in the Cocos Creator scene editor
-

7. Conclusion

AI was used as a development assistant throughout this project – primarily for explaining Cocos Creator 2.4.x-specific APIs, suggesting architectural patterns, identifying bugs, and guiding the Firebase deployment setup. It was not used to generate the project automatically or to replace the process of understanding, testing, and refining the code.

Every AI-generated suggestion was reviewed, tested in the engine, and modified to fit the project's actual requirements. The final implementation reflects decisions made after observing behavior in the editor and adjusting accordingly. The majority of the work – scene construction, asset wiring, animation configuration, map design, and manual debugging – was done without AI assistance.

The project was submitted as a working web deployment at <https://mariohw2-3d340.web.app>.